

AWS S3 Task 1

Assignment Overview

This document covers eight independent S3 tasks: creating a bucket and uploading objects, deploying a static website, enabling cross-region replication, restricting object visibility to a single Admin user via a bucket policy, configuring lifecycle rules, pushing objects via the AWS CLI, scripting bucket creation with bash, and uploading a 1 GB file via the CLI. Each task resets and is documented independently, with its own steps, validation, and troubleshooting.

The following bucket reference is used throughout this document:

Task	Bucket / Resource	Purpose
1	mylab-s3-demo-bucket-2026	Primary demo bucket; base for Tasks 1, 4, 5, 6, 8
2	mylab-static-site-bucket-2026	Static website hosting
3	mylab-s3-demo-bucket-2026 → mylab-s3-demo-bucket-dr	Cross-region replication source → destination
7	mylab-scripted-bucket-<timestamp>	Created dynamically by the bash script

Task 1: Create an S3 Bucket and Upload Objects

Outcome

A new S3 bucket named mylab-s3-demo-bucket-2026 is created with public access blocked by default, and a few sample objects are uploaded and visible in the bucket's Objects list.

Objective

To provision durable, scalable object storage and demonstrate the basic upload workflow that later tasks build on.

Prerequisites

- AWS account with console access.
- IAM permissions for S3 (s3:CreateBucket, s3:PutObject).
- Region selected (e.g., ap-south-1).
- One or two small sample files ready locally to upload.

Step-by-Step Implementation

Step 1: Open the S3 Console and Start Bucket Creation

Login to AWS Console → Search for “S3” → Open S3 Dashboard → Click “Create bucket”.

Screenshot Evidence:

Create bucket [Info](#)

Buckets are containers for data stored in S3.

General configuration

AWS Region

US East (N. Virginia) us-east-1

Bucket type [Info](#)

☒ **General purpose**

Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**

Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace

Choose the namespace where you want to create your bucket. [Learn more](#)

☒ **Global namespace**

By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**

General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [Info](#)

khadeer-demo-bucket-2026

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Step 2: Configure the Bucket

Parameter	Value
Bucket name	khadeer-demo-bucket-2026 (must be globally unique)
AWS Region	Us East(N. Virginia) us-east-1
Object Ownership	ACLs disabled (recommended)
Block Public Access settings	Keep all four options Unchecked

Click “Create bucket”.

Screenshot Evidence:

Object Ownership [Info](#)

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

☒ **ACLs disabled (recommended)**

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ **ACLs enabled**

Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership

Bucket owner enforced

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☐ **Block all public access**

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☐ **Block public access to buckets and objects granted through new access control lists (ACLs)**

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

☐ **Block public access to buckets and objects granted through any access control lists (ACLs)**

S3 will ignore all ACLs that grant public access to buckets and objects.

☐ **Block public access to buckets and objects granted through new public bucket or access point policies**

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

☐ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Turning off block all public access might result in this bucket and the objects within becoming public

AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.

Step 3: Upload Sample Objects

Open the new bucket → click “Upload” → “Add files” → select one or two sample files (e.g., sample1.txt, sample2.jpg) → click “Upload”.

Screenshot Evidence:

Upload Info

Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDKs or Amazon S3 REST API. [Learn more](#)

Drag and drop files and folders you want to upload here, or choose **Add files** or **Add folder**.

Files and folders (2 total, 25.0 B)

All files and folders in this table will be uploaded.

< 1 >

☐	Name	Folder	Type	Size
☐	khadeer2.txt	-	text/plain	16.0 B
☐	khadeer1.txt	-	text/plain	9.0 B

Validation Steps

- S3 → Buckets list shows mylab-s3-demo-bucket-2026 with status “Available”.
- The bucket's Objects list shows the uploaded files with the correct size and a recent “Last modified” timestamp.

Validation Screenshot :

Amazon S3 > Buckets > khadeer-demo-bucket-2026

khadeer-demo-bucket-2026 Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Points

Objects (3)

Copy S3 URI

Copy URL

Download

Open i

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

< 1 >

☐	Name	Type	Last modified	Size	Storage class
☐	khadeer/	Folder	-	-	-
☐	khadeer1.txt	txt	July 22, 2026, 18:13:35 (UTC+05:30)	9.0 B	Standard
☐	khadeer2.txt	txt	July 22, 2026, 18:13:34 (UTC+05:30)	16.0 B	Standard

Explanation

An S3 bucket is a globally-unique-named container that stores objects durably (11 nines of durability) within a chosen region. Keeping Block Public Access enabled by default is a security best practice, since most buckets should never be publicly readable unless a task specifically requires it (as Task 2 will). Uploading objects here demonstrates the fundamental put/list workflow that every later task in this document builds upon.

Real-Time Use Case

Organizations use S3 buckets as the default home for static assets, application data, backups, and logs — essentially the storage foundation underneath websites, data pipelines, and disaster-recovery strategies.

Common Errors and Troubleshooting

Error	Cause	Resolution
"Bucket name already exists"	S3 bucket names are globally unique across all AWS accounts	Choose a different, more unique name (e.g., append your account ID or a random suffix)
Upload fails with "Access Denied"	The IAM user/role lacks s3:PutObject permission on the bucket	Attach a policy granting s3:PutObject (e.g., AmazonS3FullAccess for a lab environment)
Bucket created in the wrong region	Region dropdown was not changed before creating the bucket	Delete the bucket and recreate it with the correct region selected (bucket region cannot be changed after creation)

Task 2: Deploy a Static Website in the S3 Bucket

Outcome

A bucket named mylab-static-site-bucket-2026 is configured for static website hosting, with an index.html and error.html uploaded, and the site is publicly reachable at its S3 website endpoint URL.

Objective

To demonstrate that S3 can serve a static website directly, without any web server or compute instance, using its built-in static website hosting feature.

Prerequisites

- AWS account with console access and S3 permissions.
- A simple index.html and error.html file prepared locally.
- Understanding that this bucket must allow public read access, unlike Task 1's bucket.

Step-by-Step Implementation

Step 1: Create a New Bucket for the Website

S3 Console → Create bucket → Name: khadeer-bucket-2026 → Region: us-east-1 → Create bucket.

Screenshot Evidence:

The screenshot shows the AWS S3 console interface for a bucket named 'khadeer-demo-bucket-2026'. The breadcrumb navigation at the top indicates the path: Amazon S3 > Buckets > khadeer-demo-bucket-2026. There are two main configuration sections visible:

- Requester pays:** This section indicates that the requester pays for requests and data transfer costs, and anonymous access is disabled. The status is 'Disabled'. An 'Edit' button is present in the top right corner of this section.
- Static website hosting:** This section indicates that the bucket is not currently configured to host a website or redirect requests. The status is 'Disabled'. It includes a recommendation to use AWS Amplify Hosting for static website hosting, with a 'Create Amplify app' button. There is also an 'Edit' button in the top right corner of this section.

Amazon S3 > Buckets > khadeer-demo-bucket-2026 > Edit static website hosting

Edit static website hosting Info

Static website hosting
Use this bucket to host a website or redirect requests. [Learn more](#)

Static website hosting
☐ Disable
☒ Enable

Hosting type
☒ Host a static website
Use the bucket endpoint as the web address. [Learn more](#)
☐ Redirect requests for an object
Redirect requests to another bucket or domain. [Learn more](#)

For your customers to access content at the website endpoint, you must make all your content publicly readable. To do so, you can edit the S3 Block Public Access settings for the bucket. For more information, see [Using Amazon S3 Block Public Access](#)

Index document
Specify the home or default page of the website.

Error document - optional

Step 2: Unblock Public Access on This Bucket

Open the bucket → Permissions tab → Block public access → Edit → uncheck “Block all public access” → Save changes → type confirm to acknowledge.

Step 3: Upload the Website Files

Objects tab → Upload → Add files → select index.html and error.html → Upload.

Step 4: Enable Static Website Hosting

Properties tab → scroll to “Static website hosting” → Edit → configure:

Parameter	Value
Static website hosting	Enable
Hosting type	Host a static website
Index document	index.html
Error document	error.html

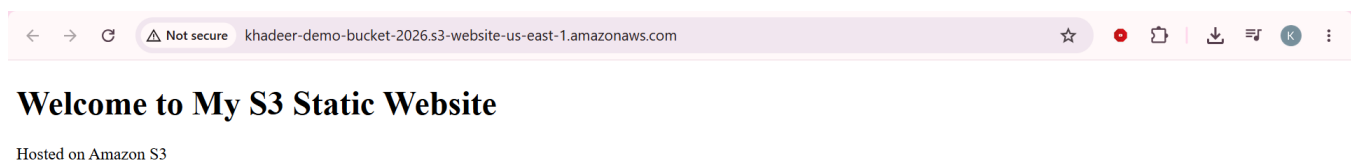
Click “Save changes” and note the Bucket website endpoint URL shown afterward.

Step 6: Access the Website

Open the Bucket website endpoint URL from Step 4 in a browser:

`http://khadeer-demo-bucket-2026.s3-website-us-east-1.amazonaws.com`

Screenshot Evidence :



Validation Steps

- The browser displays the content of index.html when visiting the website endpoint URL.
- Requesting a non-existent page on the same endpoint displays the custom error.html content instead of a generic S3 error.

Explanation

S3 static website hosting serves objects directly over HTTP from a special website endpoint, entirely without any EC2 instance or web server software. This requires two things working together: Block Public Access must be disabled for this specific bucket, and a bucket policy must explicitly grant `s3:GetObject` to everyone (Principal: `"*"`) — enabling static hosting alone does not make objects public by itself. This combination is important because both S3's object-level security and the website feature must be deliberately opted into, preventing buckets from becoming public by accident.

Real-Time Use Case

Startups and marketing teams commonly host simple static sites, landing pages, or documentation directly from S3, often placing Amazon CloudFront in front of it for HTTPS, caching, and a custom domain — avoiding the cost and management overhead of running a dedicated web server.

Common Errors and Troubleshooting

Error	Cause	Resolution
"403 Forbidden" when opening the website endpoint	Block Public Access is still enabled, or the bucket policy was not saved	Confirm Block Public Access is disabled and the public-read bucket policy is saved correctly
"NoSuchWebsiteConfiguration" error	Static website hosting was never enabled, or was enabled on the wrong bucket	Properties tab → Static website hosting → Edit → Enable, and confirm index/error document names match uploaded files exactly
Site loads but shows raw HTML as text instead of a rendered page	File uploaded without the correct Content-Type, or with a .txt extension instead of .html	Re-upload as index.html; S3 infers Content-Type text/html automatically from the .html extension

Task 3: Enable Cross-Region Replication on S3 Buckets

Outcome

A replication rule is configured on `mylab-s3-demo-bucket-2026` (source, `ap-south-1`) that automatically copies new objects to `mylab-s3-demo-bucket-dr` (destination, `us-east-1`), with the rule showing status "Active" and replicated objects appearing in the destination bucket.

Objective

To automatically maintain a copy of bucket data in a second AWS region for disaster recovery, compliance, and reduced-latency access for users in a different geography.

Prerequisites

- Task 1 completed — `mylab-s3-demo-bucket-2026` exists.
- A destination bucket created in a different region (e.g., `mylab-s3-demo-bucket-dr` in `us-east-1`).
- IAM permissions to configure replication and create the associated IAM role.

Step-by-Step Implementation

Step 1: Create the Destination Bucket in a Different Region

S3 Console → Create bucket → Name: `khadeer-shiva` → Region: US East (N. Virginia) `us-east-1` → Create bucket.

Step 2: Enable Versioning on Both Buckets

For each bucket (source and destination): open the bucket → Properties tab → Bucket Versioning → Edit → Enable → Save changes. Replication requires versioning on both buckets.

Step 3: Create the Replication Rule on the Source Bucket

Open `mylab-s3-demo-bucket-2026` → Management tab → Replication rules → Create replication rule → configure:

Parameter	Value
Rule name	CRR-to-us-east-1
Status	Enabled
Source bucket scope	Apply to all objects in the bucket
Destination bucket	khadeer-s3-shiva
IAM role	Create new role (auto-generated with the required permissions)

Click “Save”.

Step 4: Upload a Test Object to Trigger Replication

Replication only applies to objects created after the rule is enabled, so upload a new file to confirm the rule is working:

```
aws s3 cp replication-test.txt s3://khadeer-demo-bucket-2026/
```

Step 5: Verify Replication in the Destination Bucket

Open mylab-s3-demo-bucket-dr in the S3 console (switch to the us-east-1 region) and confirm replication-test.txt appears after a few minutes.

Screenshot Evidence :

```
DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls
2026-07-22 18:10:05 khadeer-demo-bucket-2026
2026-07-20 15:38:47 khadeers3
2026-07-16 19:58:08 my-vpc-flowlogs-0329-565398309825-us-east-1-an

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 cp sample.txt s3://khadeer-demo-bucket-2026/
upload: .\sample.txt to s3://khadeer-demo-bucket-2026/sample.txt

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls s3://khadeerr-demo-bucket-2026/

An error occurred (NoSuchBucket) when calling the ListObjectsV2 operation: The speci
fied bucket does not exist

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls s3://khadeer-demo-bucket-2026/
PRE khadeer/
2026-07-22 18:32:41 175 index.html
2026-07-22 18:13:35 9 khadeer1.txt
2026-07-22 18:13:34 16 khadeer2.txt
2026-07-22 19:26:52 0 sample.txt
```

Validation Steps

- The Replication rules tab on the source bucket shows CRR-to-us-east-1 with Status “Enabled”.
- mylab-s3-demo-bucket-dr contains the replicated object, and aws s3api head-object on the source object shows a Replication status of COMPLETED.

Explanation

Cross-Region Replication (CRR) asynchronously copies objects from a source bucket to a destination bucket in a different region, tracking each copy by object version — which is why versioning must be enabled on both buckets before a replication rule can be created. This is important for organizations that need geographic redundancy against a regional outage, must satisfy data-residency or compliance requirements, or want to serve a global user base from a region-local copy of the same data.

Real-Time Use Case

Global companies replicate compliance archives, media libraries, or critical application data to a secondary region so that a regional AWS outage does not make the data unavailable, and so that international users can read from a copy physically closer to them.

Common Errors and Troubleshooting

Error	Cause	Resolution
“Versioning must be enabled on both the source and destination buckets”	One or both buckets still have versioning suspended or never enabled	Enable Bucket Versioning on both buckets before creating or saving the replication rule
Objects uploaded before the rule was created do not appear in the destination	Standard replication only applies going forward, not retroactively	Use S3 Batch Replication to replicate pre-existing objects, or manually copy them with <code>aws s3 cp</code>
Replication status stuck on “PENDING” indefinitely	Normal propagation delay, or the auto-generated IAM role lacks the required replication permissions	Wait a few minutes; if it persists, review the IAM role's policy for <code>s3:ReplicateObject</code> and <code>s3:ReplicateDelete</code> permissions

Task 4: Bucket Policy So Only the Admin User Can See the Objects

Outcome

A bucket policy on `mylab-s3-demo-bucket-2026` explicitly denies `s3:GetObject` to every principal except the IAM user Admin, verified by the Admin user successfully listing objects while any other IAM user receives “Access Denied.”

Objective

To enforce strict, resource-based object visibility so that only one designated identity can view the bucket's contents, regardless of what other IAM permissions different users might otherwise hold.

Prerequisites

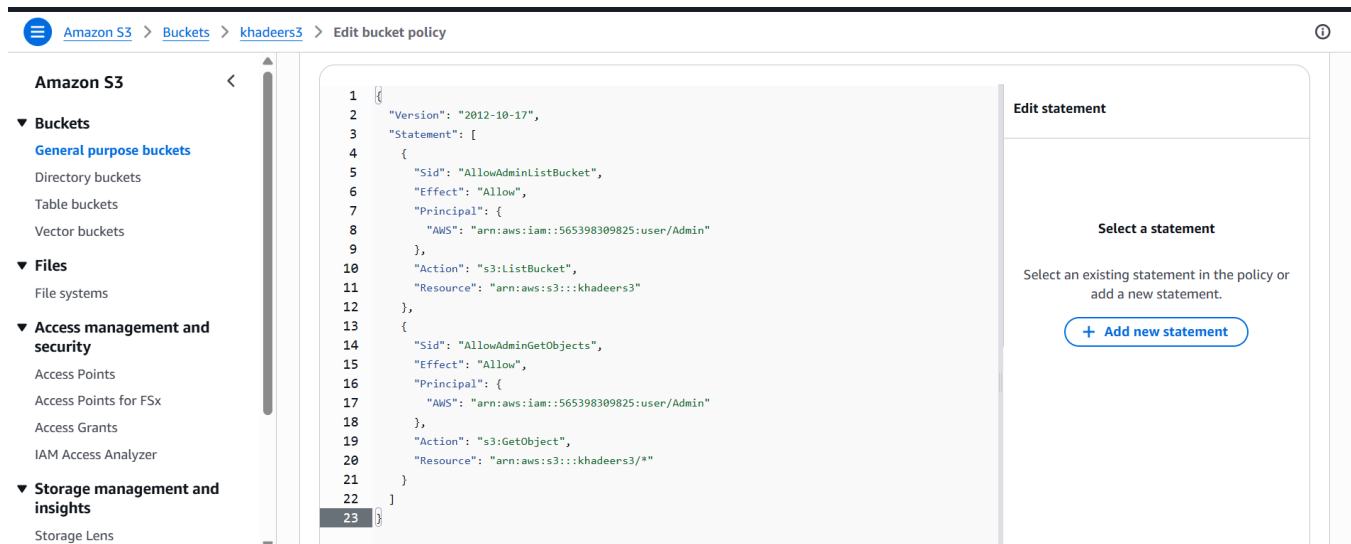
- Task 1 completed — `mylab-s3-demo-bucket-2026` exists with objects.
- An existing IAM user named Admin, and at least one other non-admin IAM user for testing.
- Permissions to edit the bucket policy.

Step-by-Step Implementation

Step 1: Note the Admin User's ARN

IAM Console → Users → Admin → copy the User ARN shown at the top of the summary (e.g., `arn:aws:iam::123456789012:user/Admin`).

Screenshot Evidence:



Step 2: Edit the Bucket Policy

S3 → mylab-s3-demo-bucket-2026 → Permissions tab → Bucket policy → Edit → paste (replacing the ARN with the Admin user's actual ARN):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyAllExceptAdmin",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mylab-s3-demo-bucket-2026/*",
      "Condition": {
        "StringNotEquals": {
          "aws:PrincipalArn": "arn:aws:iam::123456789012:user/Admin"
        }
      }
    }
  ]
}
```

Click “Save changes.”

Screenshot Evidence:

Amazon S3

Buckets

khadeers3

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

Bucket policy

Edit

Delete

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

Public access is blocked because Block Public Access settings are turned on for this bucket

To determine which settings are turned on, check your Block Public Access settings for this bucket. Learn more about [using Amazon S3 Block Public Access](#)

Copy

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAdminListBucket",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::565398309825:user/user"
      },
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::khadeers3"
    },
    {
      "Sid": "AllowAdminGetObject",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::565398309825:user/user"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::khadeers3/*"
    }
  ]
}

```

Validation Steps

- The command run with the admin-user profile lists the bucket's objects successfully.
- The identical command run with the other-user profile fails with “An error occurred (AccessDenied)”.

Explanation

This policy uses an explicit Deny combined with a StringNotEquals condition on `aws:PrincipalArn`, which denies `s3:GetObject` to every principal whose ARN does not exactly match the Admin user — and an explicit Deny in a bucket policy overrides any Allow a user might otherwise have through their own IAM permissions. This is the standard way to lock a bucket down to a single identity regardless of what broader S3 access other users or policies might separately grant.

Real-Time Use Case

Buckets holding sensitive data — financial reports, HR records, or security audit logs — are frequently restricted to a single administrative or compliance identity using this Deny-based pattern, guaranteeing that no other user, even one with broad S3 permissions elsewhere, can view the contents.

Common Errors and Troubleshooting

Error	Cause	Resolution
The Admin user is also denied access	The ARN in the policy condition does not exactly match Admin's actual ARN (typo or wrong account ID)	Run <code>aws iam get-user --user-name Admin</code> to confirm the exact ARN and update the policy
Other users can still view objects	Another statement (e.g., a broad <code>AmazonS3FullAccess</code> policy on that user) grants access, and the Deny condition was typed incorrectly so it does not match them	Double-check the condition syntax and confirm no other bucket policy statement grants broader access
Policy fails to save	JSON syntax error, such as a missing comma or brace	Use the console's built-in policy validator or a JSON linter before saving

Task 5: Set Up Lifecycle Policies to Transition or Delete Objects

Outcome

A lifecycle rule on mylab-s3-demo-bucket-2026 automatically transitions objects to S3 Standard-IA after 30 days, to S3 Glacier after 90 days, and expires (deletes) them after 365 days, visible as an enabled rule under the bucket's Management tab.

Objective

To automate storage-cost optimization and data retention by moving aging objects to cheaper storage tiers and removing them entirely once they are no longer needed, without manual intervention.

Prerequisites

- Task 1 completed — mylab-s3-demo-bucket-2026 exists.
- Permissions to configure bucket lifecycle rules.

Step-by-Step Implementation

Step 1: Open the Lifecycle Rules Section

S3 → mylab-s3-demo-bucket-2026 → Management tab → Lifecycle rules → Create lifecycle rule.

Step 2: Name the Rule and Set Its Scope

Parameter	Value
Rule name	DeleteOldFiles
Rule scope	Apply to all objects in the bucket

Screenshot Evidence:

Amazon S3 > Buckets > khadeer-demo-bucket-2026 > Lifecycle configuration > Create lifecycle rule

Create lifecycle rule [Info](#)

Lifecycle rule configuration

Lifecycle rule name

DeleteOldFiles

Up to 255 characters

Choose a rule scope

☒ Limit the scope of this rule using one or more filters

☐ Apply to all objects in the bucket

Filter type

You can filter objects by prefix, object tags, object size, or whatever combination suits your usecase.

Prefix

Add filter to limit the scope of this rule to a single prefix.

Entire Bucket

Don't include the bucket name in the prefix. Using certain characters in key names can cause problems with some applications and protocols. [Learn more](#)

Object tags

You can limit the scope of this rule to the key/value pairs added below.

[Add tag](#)

Step 3: Configure Storage Class Transitions

Check “Move current versions of objects between storage classes” and add:

Parameter	Value
Transition 1	30 days after object creation → Standard-IA
Transition 2	90 days after object creation → Glacier

Screenshot Evidence:

Amazon S3 > Buckets > khadeer-demo-bucket-2026 > Lifecycle configuration > Create lifecycle rule

Lifecycle rule actions

Choose the actions you want this rule to perform.

- ☒ Transition current versions of objects between storage classes
This action will move current versions.
- ☐ Transition noncurrent versions of objects between storage classes
This action will move noncurrent versions.
- ☐ Expire current versions of objects
- ☐ Permanently delete noncurrent versions of objects
- ☐ Delete expired object delete markers or incomplete multipart uploads
These actions are not supported when filtering by object tags or object size.

Transitions are charged per request
For a lifecycle transition action, each request corresponds to an object transition. For details on lifecycle transition pricing, see requests pricing info on the [Storage & requests](#) tab of the [Amazon S3 pricing page](#).

☒ I acknowledge that this lifecycle rule will incur a transition cost per request.

By default, objects less than 128KB will not transition across any storage class
We don't recommend transitioning objects less than 128 KB because the transition costs can outweigh the storage savings. If your use case requires transitioning objects less than 128 KB, specify a minimum object size filter for each applicable lifecycle rule with a transition action.

Transition current versions of objects between storage classes

Choose transitions to move current versions of objects between storage classes based on your use case scenario and performance access requirements. These transitions start from when the objects are created and are consecutively applied. [Learn more](#)

Choose storage class transitions	Days after object creation	
Standard-IA	30	<button>Remove</button>
Glacier Deep Archive	60	<button>Remove</button>

Validation Steps

- The Lifecycle rules tab lists Archive-and-Expire-Rule with Status “Enabled” and the correct transition/expiration summary.
- Selecting the rule shows the exact transition days (30, 90) and expiration day (365) configured in the previous steps.

Validation Screenshot:

Lifecycle configuration

To manage your objects so that they are stored cost effectively throughout their lifecycle, configure their lifecycle. A lifecycle configuration is a set of rules that define actions that Amazon S3 applies to a group of objects. Lifecycle rules run once per day.

Default minimum object size for transitions
All storage classes 128K

Lifecycle rules (1)

Use lifecycle rules to define actions you want Amazon S3 to take during an object's lifetime such as transitioning objects to another storage class, archiving them, or deleting them after a specified period of time. [Learn more](#)

Find lifecycle rules by name

Lifecycle rule name	Status	Scope	Current version actions	Noncurrent versions ac...	Expired object delete ...
DeleteOldFiles	Enabled	Filtered	Transition to Standard-IA, then	-	-

Explanation

Lifecycle rules let S3 automatically move objects to progressively cheaper, lower-access storage classes as they age, and delete them outright once they pass their retention period — all evaluated by a background process without any application or administrator needing to run a manual cleanup job. This balances storage cost against retrieval speed and access frequency over an object's lifetime, and it enforces retention policy consistently rather than relying on someone remembering to delete old data.

Real-Time Use Case

Log files and backups are commonly moved to Glacier after a month and deleted after a year to satisfy audit or compliance retention requirements while minimizing ongoing storage costs — a standard data lifecycle management practice used broadly across enterprises.

Common Errors and Troubleshooting

Error	Cause	Resolution
Rule appears configured but objects have not moved yet	Lifecycle transitions run on a scheduled daily basis and only take effect once an object reaches the configured age from its creation date	This is expected behavior — wait for the object to reach the configured age; there is no way to force an immediate transition
“Transition to STANDARD_IA requires a minimum of 30 days” validation error	S3 enforces a minimum eligible storage duration before certain storage-class transitions	Set the transition day count to at least the minimum required by the target storage class
Rule conflicts with an existing rule on the same objects	Two lifecycle rules have overlapping prefixes/filters	Adjust the rule scope/prefix so rules do not overlap, or consolidate them into a single rule

Task 6: Push Objects to S3 Using the AWS CLI

Outcome

Files and an entire folder are uploaded to mylab-s3-demo-bucket-2026 using AWS CLI commands, confirmed by listing the bucket's contents from the command line.

Objective

To perform S3 upload operations programmatically instead of through the console, forming the basis for scripted or automated data transfer.

Prerequisites

- Task 1 completed — mylab-s3-demo-bucket-2026 exists.
- AWS CLI installed and configured (aws configure) with S3 permissions.

Step-by-Step Implementation

Step 1: Upload a Single File

```
aws s3 cp sample.txt s3://khadeer-demo-bucket-2026/
```

Screenshot Evidence:

```

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls
2026-07-22 18:10:05 khadeer-demo-bucket-2026
2026-07-20 15:38:47 khadeers3
2026-07-16 19:58:08 my-vpc-flowlogs-0329-565398309825-us-east-1-an

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 cp sample.txt s3://khadeer-demo-bucket-2026/
upload: .\sample.txt to s3://khadeer-demo-bucket-2026/sample.txt

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls s3://khadeerr-demo-bucket-2026/

An error occurred (NoSuchBucket) when calling the ListObjectsV2 operation: The speci
fied bucket does not exist

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls s3://khadeer-demo-bucket-2026/
PRE khadeer/
2026-07-22 18:32:41 175 index.html
2026-07-22 18:13:35 9 khadeer1.txt
2026-07-22 18:13:34 16 khadeer2.txt
2026-07-22 19:26:52 0 sample.txt

```

Validation Steps

- The `aws s3 ls --recursive` output lists `sample.txt` and every file under `my-folder/` with the correct sizes.
- The S3 console's Objects list for the bucket matches the same files.

Explanation

The `aws s3 cp` and `aws s3 sync` commands wrap the underlying S3 `PutObject` API calls, enabling scripted, repeatable uploads instead of manual console clicks. `sync` specifically compares the source and destination and transfers only new or changed files, which is efficient for repeated updates to the same folder over time, unlike `cp` which always transfers the specified file regardless of whether it already exists remotely.

Real-Time Use Case

CI/CD pipelines commonly push build artifacts, static site files, or application logs to S3 via the CLI as an automated deployment or logging step, rather than requiring a person to manually upload files through the console.

Common Errors and Troubleshooting

Error	Cause	Resolution
"Unable to locate credentials"	The AWS CLI has not been configured with valid credentials	Run <code>aws configure</code> and supply a valid Access Key ID and Secret Access Key
"An error occurred (AccessDenied) when calling the PutObject operation"	The IAM user/role lacks <code>s3:PutObject</code> permission on the target bucket	Attach a policy granting <code>s3:PutObject</code> on the bucket, or use a role with broader S3 access for the lab
<code>aws s3 sync</code> appears to skip files	Files are unchanged (same size and timestamp) since the last sync, which is expected sync behavior	Use <code>aws s3 cp</code> for a guaranteed re-upload, or modify the files before syncing again

Task 7: Write a Bash Script to Create an S3 Bucket

Outcome

Running a bash script creates a new, uniquely-named S3 bucket (e.g., `mylab-scripted-bucket-1737550000`) in the specified region, confirmed via the AWS CLI without any manual console steps.

Objective

To automate bucket provisioning through a script for repeatability and consistency, forming a reusable building block for larger infrastructure-automation workflows.

Prerequisites

- AWS CLI installed and configured with S3 permissions.
- A bash shell (Linux, macOS, or WSL on Windows).

Step-by-Step Implementation

Step 1: Create the Script File

```
vi create_s3_bucket.sh
```

Screenshot Evidence (Mandatory) — *insert screenshot here*

Step 2: Write the Script

```
#!/bin/bash
BUCKET_NAME="khadeer-demo-bucket-2026"
REGION="us-east-1"

# Display message
echo "Creating bucket..."

# Create bucket
If["$REGION" = "us-east-1"]
aws s3api create-bucket \
  --bucket $BUCKET_NAME \
Else
aws s3api create-bucket \
  --bucket $BUCKET_NAME \
  --region $REGION \
  --create-bucket-configuration LocationConstraint=$REGION

# Display success message
echo "Bucket created successfully!"
```

Note: the `--create-bucket-configuration` flag should be omitted entirely when `REGION` is `us-east-1`, since that region is AWS's default and does not accept an explicit `LocationConstraint`.

Screenshot Evidence:

MINGW64:/c/Users/DELL/Desktop

#!/bin/bash

BUCKET_NAME="khadeer-demo-bucket-2026"

REGION="us-east-1"

echo "Creating bucket..."

if ["\$REGION" = "us-east-1"]; then

aws s3api create-bucket \
--bucket \$BUCKET_NAME

else

aws s3api create-bucket \
--bucket \$BUCKET_NAME \
--region \$REGION \
--create-bucket-configuration LocationConstraint=\$REGION

fi

echo "Bucket Created."

~

Step 3: Make the Script Executable and Run It

```
chmod +x create_s3_bucket.sh  
./create_s3_bucket.sh
```

Screenshot Evidence:

```
DELL@KHADEER MINGW64 ~/Desktop  
$ vi create_bucket.sh  
  
DELL@KHADEER MINGW64 ~/Desktop  
$ chmod +x create_bucket.sh  
  
DELL@KHADEER MINGW64 ~/Desktop  
$ aws s3 ls  
2026-07-22 18:10:05 khadeer-demo-bucket-2026  
2026-07-20 15:38:47 khadeers3  
2026-07-16 19:58:08 my-vpc-flowlogs-0329-565398309825-us-east-1-an  
  
DELL@KHADEER MINGW64 ~/Desktop  
$ ./create_bucket.sh  
{  
  "Location": "http://khadeer-bucket-0329.s3.amazonaws.com/",  
  "BucketArn": "arn:aws:s3:::khadeer-bucket-0329"  
}  
Bucket created successfully!
```

Step 4: Verify the Bucket Was Created

```
aws s3 ls
```

Screenshot Evidence:

```

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls
2026-07-22 18:10:05 khadeer-demo-bucket-2026
2026-07-20 15:38:47 khadeers3
2026-07-16 19:58:08 my-vpc-flowlogs-0329-565398309825-us-east-1-an

DELL@KHADEER MINGW64 ~/Desktop
$ ./create_bucket.sh
{
  "Location": "http://khadeer-bucket-0329.s3.amazonaws.com/",
  "BucketArn": "arn:aws:s3:::khadeer-bucket-0329"
}
Bucket created successfully!

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls
2026-07-22 19:51:23 khadeer-bucket-0329
2026-07-22 18:10:05 khadeer-demo-bucket-2026
2026-07-20 15:38:47 khadeers3
2026-07-16 19:58:08 my-vpc-flowlogs-0329-565398309825-us-east-1-an

```

Validation Steps

- The script prints “S3 bucket mylab-scripted-bucket-<timestamp> created in ap-south-1.” with no errors.
- aws s3 ls (or the S3 console) shows the newly created bucket with the timestamped name.

Explanation

The script uses a Unix timestamp appended to the bucket name to guarantee global uniqueness on every run, avoiding the “BucketAlreadyExists” conflict that would occur with a fixed, hardcoded name. Wrapping create-bucket in a script like this is the basis of Infrastructure-as-Code style resource provisioning, and the same pattern can easily be extended into a larger deployment script or integrated into a CI/CD pipeline.

Real-Time Use Case

Teams that need a bucket per environment or per customer/tenant automate bucket creation this way rather than manually creating each one through the console, ensuring naming conventions and configuration stay consistent across every bucket created.

Common Errors and Troubleshooting

Error	Cause	Resolution
“BucketAlreadyExists”	An extremely rare timestamp collision, or the script was modified to use a fixed name	Rerun the script (a new timestamp will be generated), or confirm the dynamic naming logic was not removed
“An error occurred (IllegalLocationConstraintException)”	The --region flag and the LocationConstraint value do not match, or LocationConstraint was supplied for us-east-1	Ensure --region and LocationConstraint use the same region code, and omit LocationConstraint entirely when the region is us-east-1
“AccessDenied” when creating the bucket	The configured CLI credentials lack s3:CreateBucket permission	Configure the CLI with credentials belonging to an identity that has sufficient S3 permissions

Task 8: Upload a 1 GB File to S3 Using the CLI

Outcome

A 1 GB test file is uploaded to mylab-s3-demo-bucket-2026 using the AWS CLI, which automatically performs a multipart upload in the background, and the file appears in the bucket with the correct size.

Objective

To demonstrate handling large file uploads with the AWS CLI and understand how multipart upload makes large-object transfers reliable and efficient.

Prerequisites

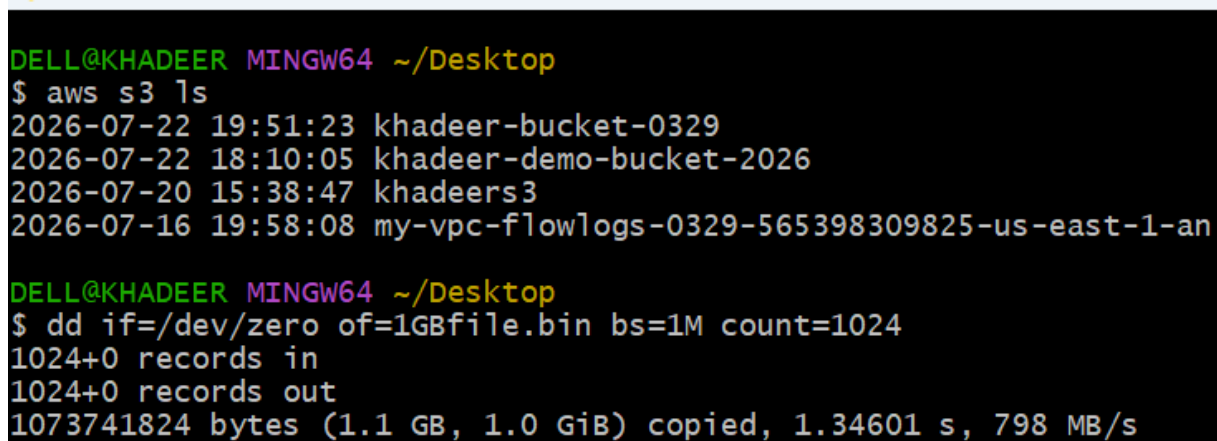
- Task 1 completed — mylab-s3-demo-bucket-2026 exists.
- AWS CLI installed and configured with S3 permissions.
- At least 1 GB of free local disk space to generate the test file.

Step-by-Step Implementation

Step 1: Generate a 1 GB Test File

```
dd if=/dev/zero of=testfile-1gb.bin bs=1M count=1024
```

Screenshot Evidence:



```
DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls
2026-07-22 19:51:23 khadeer-bucket-0329
2026-07-22 18:10:05 khadeer-demo-bucket-2026
2026-07-20 15:38:47 khadeers3
2026-07-16 19:58:08 my-vpc-flowlogs-0329-565398309825-us-east-1-an

DELL@KHADEER MINGW64 ~/Desktop
$ dd if=/dev/zero of=1GBfile.bin bs=1M count=1024
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 1.34601 s, 798 MB/s
```

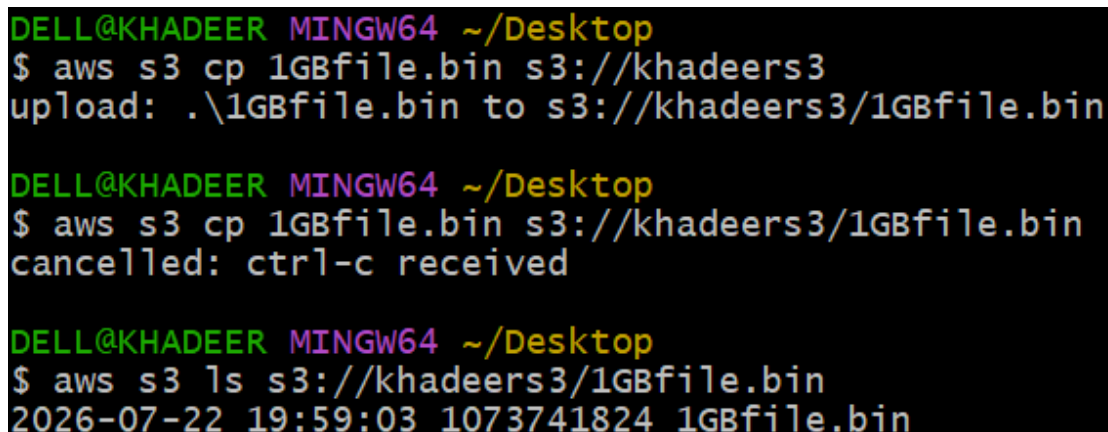
Step 2: Upload the File via the CLI

```
aws s3 cp testfile-1gb.bin s3://khadeer3
```

Step 4: Verify the Upload

```
aws s3 ls s3://khadeer/1GBfile.bilarge
```

Screenshot Evidence:



```
DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 cp 1GBfile.bin s3://khadeers3
upload: .\1GBfile.bin to s3://khadeers3/1GBfile.bin

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 cp 1GBfile.bin s3://khadeers3/1GBfile.bin
cancelled: ctrl-c received

DELL@KHADEER MINGW64 ~/Desktop
$ aws s3 ls s3://khadeers3/1GBfile.bin
2026-07-22 19:59:03 1073741824 1GBfile.bin
```

Validation Steps

- The upload completes with a “upload:” confirmation line and no errors, despite the large file size.
- `aws s3 ls --human-readable` shows `testfile-1gb.bin` with a size of approximately 1.0 GiB.

Explanation

The AWS CLI automatically splits uploads larger than a default threshold (8 MB) into multiple parts, uploads them in parallel, and reassembles them server-side into a single object — all transparently, without any extra flags needed for this task. This matters because a single 1 GB PUT request over an unreliable network could fail entirely on a brief interruption, whereas multipart upload can retry individual failed parts without restarting the whole transfer, making large-file uploads (backups, videos, VM images) far more resilient.

Real-Time Use Case

Enterprises uploading large backup archives, media files, or VM/AMI export images to S3 rely on this same built-in multipart upload behavior in the CLI and SDKs to transfer large objects reliably, without needing to implement chunking logic themselves.

Common Errors and Troubleshooting

Error	Cause	Resolution
Upload appears to hang or takes a long time	Expected behavior for a 1 GB transfer depending on available network bandwidth	Allow the transfer to continue; avoid cancelling prematurely, since the CLI shows a progress indicator during the upload
Upload fails partway with a network-related error	A network interruption occurred during a part transfer	The CLI automatically retries failed parts; if the overall command still fails, simply rerun the same <code>aws s3 cp</code> command
“No space left on device” while creating the test file	Insufficient free local disk space for a 1 GB file	Run <code>df -h</code> to confirm free space beforehand, or generate a smaller test file and note the size difference

